Workshop 2 - Part 2

Design Artifacts and System Modeling

Submitted by:
Carlos Daniel Albarracin Cruz - caalbarracin@unal.edu.co
Nicolas Aguirre Velasquez - niaguirrev@unal.edu.co
David Nickolai Parra Ariza - dparraar@unal.edu.co
Gabriel Cuellar Garzon - gacuellar@unal.edu.co

Professor:
Liliana Marcela Olarte Mesa
lmolartem@unal.edu.co

Friday, October 30th



National University of Colombia
Faculty of Engineering
Computer Systems Engineering
Software Engineering II
2026

Table of Content

Workshop 2 – Part 2 1

1. Business Model Processes 3
2. Architecture Diagram 6
3. References 8
4. Use of Artificial Intelligence 8

1. Business Model Processes

- Core process: *Clinical consultation, from appointment request to prescription and stock deduction*

-Role in the application. This process is the main business flow of VetCare because it crosses the four modules of the system. It starts in Space & Appointment Management with the booking of the visit, continues in Patient & Client Management with the clinical record, the prescription and the vaccination, and ends in Pharmacy & Inventory Management with the stock deduction and the alerts. Access Control determines which role performs each step. Modeling it shows how the data and the responsibilities of each module connect in a single operation of the clinic.

-Participants. The process involves the Pet Owner, the Receptionist, the Veterinarian, the Administrator and the VetCare System, which are the lanes of the clinic pool. The Messaging Service is an external participant that delivers the automated messages.

-Description of the process. The process starts when the Pet Owner needs care and requests an appointment by selecting a pet, a service type and a time slot. The system registers the request as pending and notifies the reception team. The Receptionist selects a consultation room and a veterinarian, and the system validates their availability within the working hours of the veterinarian. If the slot is not available, the system suggests the next available slots and the Receptionist selects again. Otherwise, the Receptionist confirms the appointment, and the system sets its status to “confirmed” and updates the available slots.

After the confirmation, the process continues in parallel. The system sends a confirmation message and, when the date of the appointment approaches, a reminder message, both through the Messaging Service. The Pet Owner receives both messages and attends the appointment.

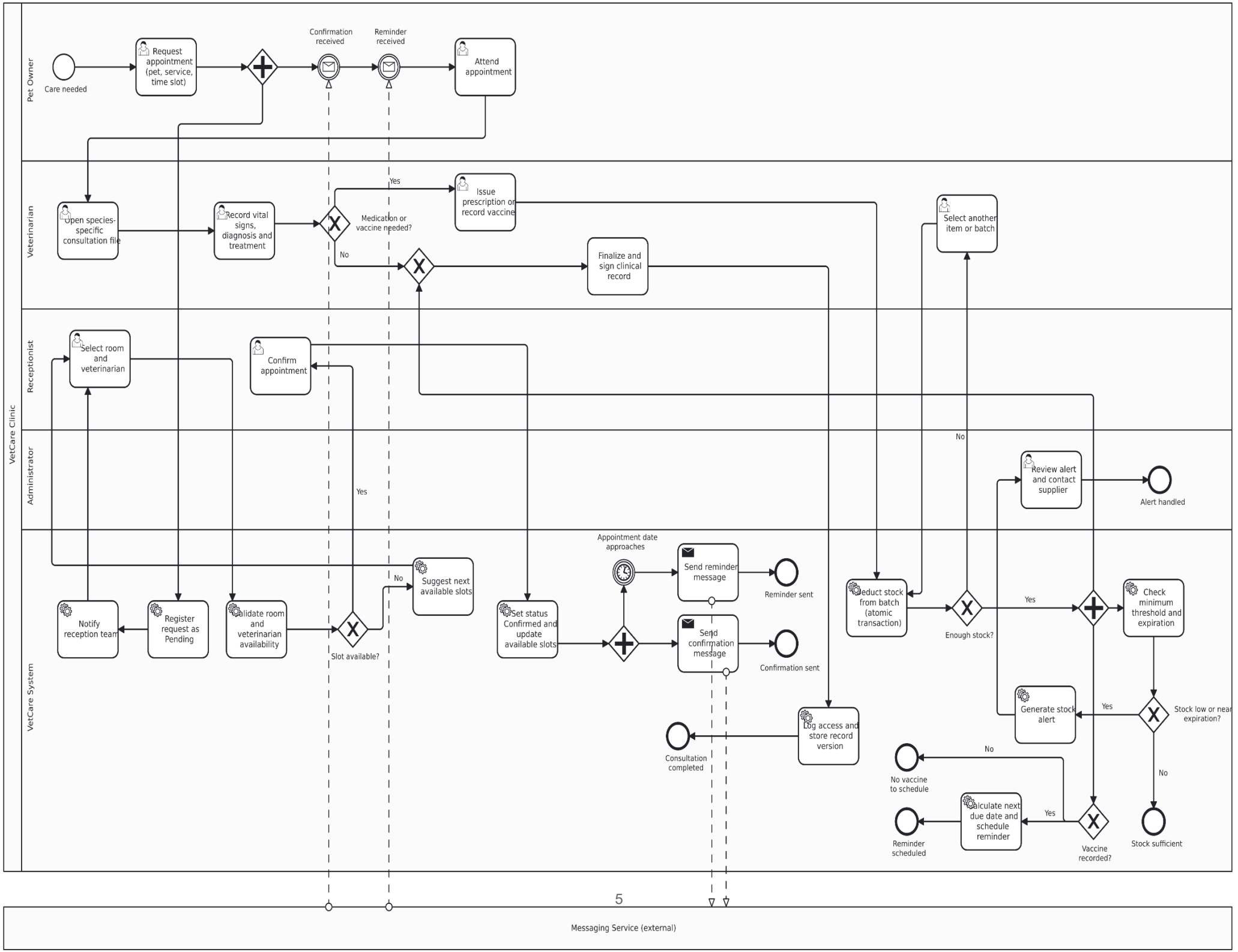
During the consultation, the Veterinarian opens the consultation file with the fields specific to the species of the patient and records the vital signs, the diagnosis and the treatment. If medication or a vaccine is needed, the Veterinarian issues the prescription or records the vaccine by selecting the corresponding inventory item. The system then deducts the quantity from the batch. If the stock is not enough, the Veterinarian selects another item or batch and the deduction is repeated.

When the deduction succeeds, three things happen in parallel. First, the system checks the minimum threshold and the expiration date of the item. If the item is low or near expiration, it generates a stock alert, and the Administrator reviews the alert and contacts the supplier. Second, if a vaccine was recorded, the system calculates its next due date and schedules the reminder. Third, the consultation continues to its closing.

The Veterinarian finalizes and signs the clinical record. The system logs the access and stores the record as a new version that preserves the previous one. The process ends with the consultation completed.

-Scope. The cancellation and rescheduling of appointments, the hospitalization of patients and the purchase of stock are modeled as separate flows and are not part of this diagram. Billing is outside the scope of the project.

-Notation. The diagram follows BPMN 2.0 and was created with bpmn.io. Exclusive gateways (X) represent decisions, parallel gateways (+) represent activities that run at the same time, and dashed lines represent the messages exchanged with the external Messaging Service.


VetCare Clinic
Pet Owner
Veterinarian
Receptionist
Administrator
VetCare System
Care needed
Request appointment (pet, service, time slot)
Confirmation received
Reminder received
Attend appointment
Open species-specific consultation file
Record vital signs, diagnosis and treatment
Medication or vaccine needed?
Yes
No
Issue prescription or record vaccine
Finalize and sign clinical record
Select another item or batch
Select room and veterinarian
Confirm appointment
Yes
No
Review alert and contact supplier
Alert handled
Notify reception team
Register request as Pending
Validate room and veterinarian availability
Slot available?
Suggest next available slots
Set status Confirmed and update available slots
Appointment date approaches
Send reminder message
Reminder sent
Send confirmation message
Confirmation sent
Consultation completed
Log access and store record version
Deduct stock from batch (atomic transaction)
Enough stock?
Check minimum threshold and expiration
Generate stock alert
Stock low or near expiration?
No vaccine to schedule
Calculate next due date and schedule reminder
Reminder scheduled
Vaccine recorded?
Stock sufficient
Messaging Service (external)
5

2. Architecture Diagram

The selected architecture was the *architecture by layers* because it clearly and easily separates technical responsibilities. By isolating the graphical interface, business rules, and data access, the code becomes easier to maintain, test, and scale. This separation ensures that if the database is changed in the future or a mobile application is developed, the clinical and administrative logic will remain intact. Additionally, it centralizes the application of security and Role-Based Access Control (RBAC) before handling sensitive medical information.

Presentation Layer: Frontend (React)
This is the interface that the different actors (Owner, Receptionist, Vet, Vet Assistant, Clinic Admin) interact with. Its sole function is to render information and capture user events.

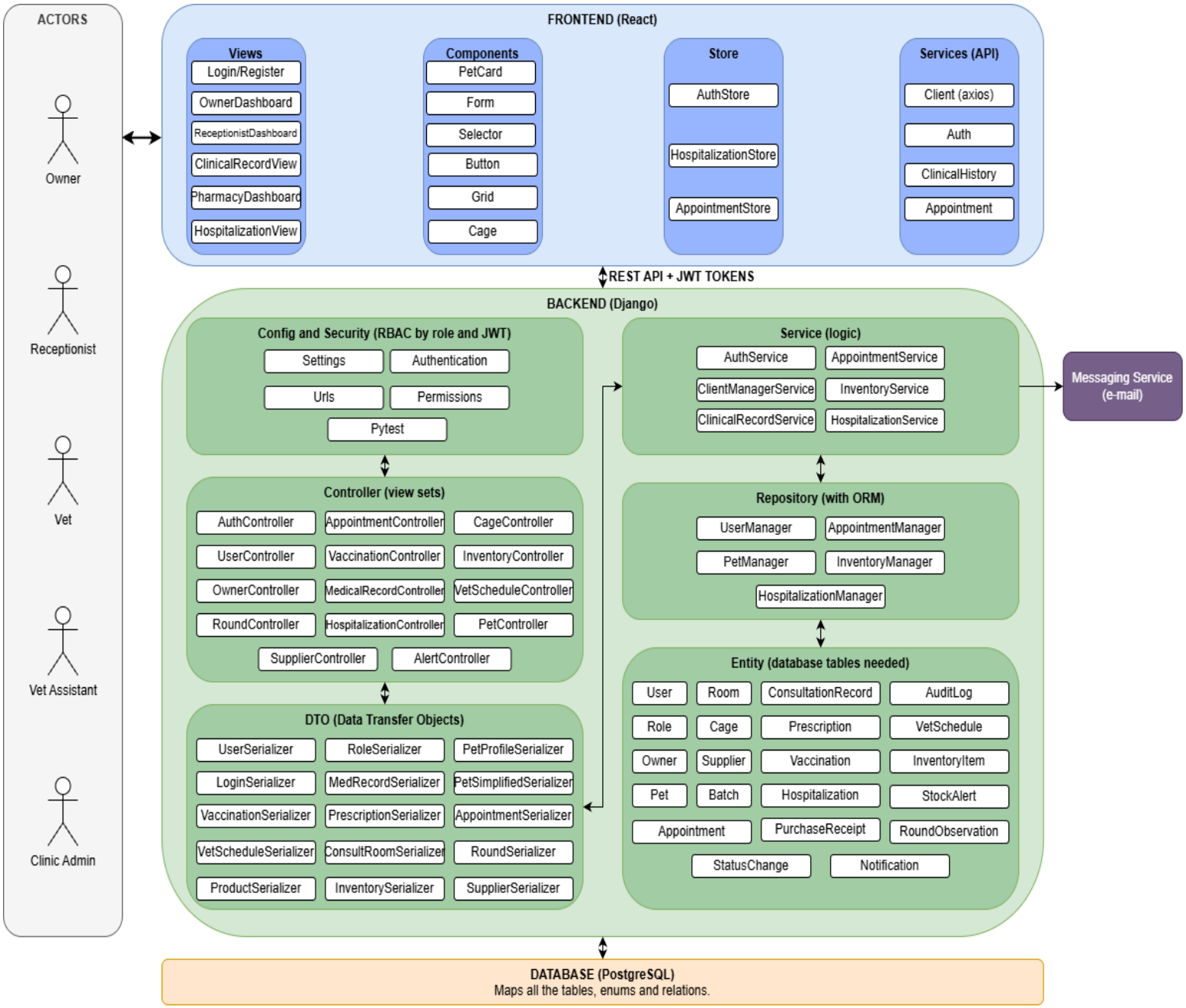
- Views: Contains complete screens like OwnerDashboard, ReceptionistDashboard, and ClinicalRecordView. Their existence is justified in organizing the specific user experience required by each role.
- Components: Groups reusable visual elements such as PetCard, Form, Selector, Grid, and Cage. They are structured this way to avoid duplicating interface code throughout the project.
- Store: Manages the global state in the browser through files like AuthStore, HospitalizationStore, and AppointmentStore. It is used so the application can react without reloading the page.
- Services (API): Encapsulates HTTP calls using the axios library in files like Client, Auth, or Appointment. Its purpose is to separate the network logic (REST API and JWT Token sending) from the visual components.

Application Layer: Backend (Django)
This is the central engine of the system that processes the business rules, validating every transaction under a REST API.

- Config and Security: Centralizes Settings, Urls, Authentication, Permissions, and Pytest. This upper layer intercepts traffic to validate the JWT token and apply RBAC, denying unauthorized actions from the start.
- Controller (view sets): Includes route controllers such as AuthController, AppointmentController, or MedicalRecordController. Its only responsibility is to receive HTTP requests, send them to the services, and return the response; it acts as a traffic director with no medical logic.
- DTO (Data Transfer Objects): Uses serializers like UserSerializer, PetProfileSerializer, PetSimplifiedSerializer, or InventorySerializer. It serves to translate complex database objects into plain text (JSON) and hides sensitive fields depending on the actor making the request.
- Service (logic): Contains the intelligence of the system in files like AppointmentService, InventoryService, and HospitalizationService. This separation is vital to calculate availability, avoid appointment overlaps, or process inventory deductions without depending on the web environment.
- Repository (with ORM): Encapsulates database queries through managers like UserManager, AppointmentManager, or InventoryManager. It is justified to avoid writing SQL code directly and allow the service to operate exclusively with objects.

- Entity: Defines the necessary database tables in models like User, Role, Owner, Pet, Appointment, ConsultationRecord, Batch, or StockAlert. It guarantees the data structure and its relations at the source code level.

Data Layer: Database (PostgreSQL)
This is the relational database engine that maps all the tables, enumerators (enums), and relations defined by the entities. It securely centralizes all the persistence of the veterinary clinic's records.



3. References

Albarracin Cruz, C. D., Aguirre Velásquez, N., Parra Ariza, D. N., & Cuéllar Garzón, G. (2026). *Workshop 2: Design artifacts and system modeling* [Course deliverable]. Software Engineering II, Universidad Nacional de Colombia.

bpmn.io. (n.d.). *bpmn.io* [BPMN modeling tool]. https://bpmn.io/

diagrams.net. (n.d.). *draw.io* [Diagramming tool]. https://draw.io/

Richards, M. (2015). *Software architecture patterns*. O'Reilly Media.

4. Use of Artificial Intelligence

The team used AI assistants (Gemini, by Google, and Claude, by Anthropic) as support tools during the preparation of this workshop. Their use was limited to the following tasks:

- Reviewing the content of the document to identify errors and inconsistencies.
- Translating the text into English.
- Drafting the first models of the BPMN diagram and of the software architecture.